Fundamentals of Artificial Intelligence - Lecture 04

Lecture overview

Lecture 04 establishes the theoretical foundation for informed search algorithms in artificial intelligence, contrasting them with uninformed strategies like Uniform Cost Search (UCS). While UCS guarantees optimality by expanding nodes based solely on accumulated path cost $g(n)$, it explores indiscriminately. Informed search introduces heuristics $h(n)$ to estimate proximity to the goal, exemplified by Manhattan and Euclidean distances in route-finding and grid-world scenarios. Greedy Best-First Search (GBFS) utilizes only $h(n)$ to prioritize nodes appearing closest to the goal; however, this "greedy" approach is neither complete nor optimal, as it may overlook lower-cost paths or get trapped in local minima. Beam Search offers a memory-efficient alternative by retaining only the top k nodes at each level, though it sacrifices completeness and optimality due to aggressive pruning.

The lecture culminates in the introduction of A search, which combines UCS and GBFS by evaluating nodes using $f(n) = g(n) + h(n)$. A guarantees an optimal solution in tree search if the heuristic is admissible (never overestimating the true cost), a property often achieved by solving relaxed versions of the problem, such as ignoring obstacles in the 8-puzzle. For graph search involving loop-breaking via a closed set, optimality requires the stronger condition of consistency, where the heuristic satisfies the triangle inequality to ensure $f(n)$ values never decrease along a path. While A is powerful and widely applied in domains ranging from robotics to language analysis, it is memory-intensive, prompting the discussion of space-efficient variants like IDA, RBFS, and SMA*.

Slide 1: Fundamentals of Artificial Intelligence

This lecture introduces the fundamentals of artificial intelligence, specifically focusing on informed search algorithms. Presented by Professor Tias Guns in Lecture 04 of the 2024-25 academic year, the session draws its core material from CS188 at Berkeley and Professor De Raedt's AI course. The content establishes the theoretical basis for informed search strategies within the broader context of artificial intelligence.

Slide 2: Example: route-finding in Romania

Romania route finding models cities as states and roads as transitions with distances as costs. The standard example starts at Arad and seeks Bucharest, allowing search strategies to be compared on one map.

Slide 3: Uniform Cost Search

Uniform Cost Search (UCS) expands nodes based on the lowest accumulated path cost from the start, ignoring any information about the goal's location. While this strategy causes UCS to explore options in every direction equally, it guarantees that the algorithm is both complete and optimal, always finding the least-cost solution regardless of whether edge costs are uniform or variable.

Slide 4: What we would like to have happen

The objective is to guide the search process directly toward the goal rather than exploring indiscriminately. This slide contrasts uninformed search, which lacks direction, with informed search, which utilizes specific guidance to navigate from the start state to the goal efficiently.

Slide 5: Search Heuristics

A search heuristic is a problem-specific function that estimates how close a state is to a goal, guiding the search algorithm toward the target. In path planning, common examples include Manhattan distance (sum of horizontal and vertical steps) and Euclidean distance (straight-line distance). These functions provide computable values to prioritize node expansion, even though they may ignore obstacles like walls and thus do not represent feasible plans.

Slide 6: Greedy Best-First Search

In the context of a sliding puzzle, valid actions involve physically moving an adjacent tile into the empty space, which is equivalent to shifting the empty space itself. To solve the puzzle with the fewest moves possible, each action incurs a uniform step cost. This framework defines the physical constraints and cost structure necessary for applying search algorithms like Greedy Best-First Search.

Slide 7: Greedy Best-First Search

Greedy Best-First Search expands the frontier node with the lowest heuristic value, typically using Euclidean distance to the goal, to select the path that appears closest at each step. While this "greedy" approach looks only one hop ahead and can find solutions quickly in constrained environments like mazes, it is not optimal. As demonstrated by the Romania map example, the resulting path to Bucharest is often not the shortest possible route because minimizing local distance does not guarantee a minimal total path cost.

Slide 8: Greedy Best-First Search (GBFS)

Greedy Best-First Search (GBFS) expands the node on the frontier estimated to be closest to a goal state using only the heuristic cost, h(n). While it often proceeds directly to a solution, its worst-case behavior resembles a poorly guided Depth-First Search. Unlike A*, GBFS ignores backward costs and adds all generated states to the frontier, which can cause the frontier size to grow very large.

Slide 9: Video of Demo Contours GBFS (Empty)

This embedded demo shows Greedy Best-First Search contours in an empty layout. A contour represents the search order induced by the heuristic; it helps visualize where nodes are expanded before the goal.

Slide 10: Video of Demo Contours GBFS (Pacman Small Maze)

The Pacman maze demo shows how a heuristic guides Greedy Best-First Search through the maze. The search expands states that appear closer to the goal, which can be fast but does not account for the full path cost.

Slide 11: Beam Search

Beam Search is a variant of Breadth-First and Greedy Best-First search that performs level-wise expansion while maintaining strict memory management. At each depth, it expands all nodes but retains only the k best paths on the frontier based on their heuristic value, aggressively deleting all others without tracking them. This approach ensures efficient resource usage but may fail to find a solution if the optimal path is pruned, potentially requiring developers to re-run the algorithm with a larger k.

Slide 12: Beam Search Properties, beam size k

Beam Search limits memory by maintaining a frontier queue of size k, though expansion requires $O(kb)$ space. It expands $O(skb)$ nodes if a solution is found at depth s, or $O(mkb)$ otherwise, where m is the maximum tree depth. Because it discards paths to enforce the budget, Beam Search is neither complete nor optimal; setting $k=1$ reduces it to greedy hill-climbing search without backtracking.

Slide 13: Informed search...

Greedy best-first search (GBFS) lacks optimality, prompting the question of whether an informed search method can combine GBFS's approach with the optimality guarantees found in Uniform Cost Search (UCS). This slide highlights the need for a strategy that maintains heuristic guidance while ensuring optimal solutions.

Slide 14: A* Search

A evaluates a node with f(n) = g(n) + h(n), where g(n) is the path cost already paid and h(n) estimates the remaining cost to a goal. It expands the lowest estimated total-cost node. With nonnegative step costs, an admissible heuristic, and correct graph-search handling, A returns an optimal path; an overestimating heuristic can forfeit that guarantee.

Slide 15: Combining UCS and Greedy

A Search combines Uniform-Cost Search (UCS) and Greedy Best-First Search by ordering nodes based on the sum of backward path cost $g(n)$ and forward heuristic cost $h(n)$, defined as $f(n) = g(n) + h(n)$. While UCS prioritizes only $g(n)$ and Greedy prioritizes only $h(n)$, A uses this combined value to determine node priority within a priority queue. This approach allows the algorithm to balance actual path costs with estimated goal proximity when selecting which node to expand next.

Slide 16: A*: the core idea

A expands the node most likely to be on an optimal path by selecting the one with the lowest value of $g(n) + h^(n)$, where $g(n)$ is the cost from the root to n and $h^(n)$ is the true optimal cost from n to the closest goal. Since $h^(n)$ is typically unknown, A* uses a heuristic approximation $h(n)$ within a tree search algorithm that employs a priority queue ordered by $f(n) = g(n) + h(n)$.

Slide 17: Example: route-finding in Romania

This slide illustrates route-finding in Romania using Euclidean distance to Bucharest as the heuristic function h(n). Starting from Arad (266 km), the algorithm expands the node to evaluate neighbors Sibiu, Timisoara, and Zerind. By comparing their respective distances to the goal, the best-first search selects Sibiu (253 km) as the closest node on the frontier for subsequent expansion.

Slide 18: Example: pathing in Pacman

In Pacman pathing, the heuristic $h(n)$ uses Manhattan distance ($|\Delta x| + |\Delta y|$) rather than straight-line Euclidean distance because Pacman cannot move diagonally. While Euclidean distance yields a smaller value (e.g., 11.2), Manhattan distance provides a more accurate estimate of the actual steps required (e.g., 15) by aligning with the agent's restricted up, down, left, and right actions. This heuristic guides the search to expand nodes with the smallest estimated cost first, prioritizing paths that correspond to valid movements in the grid world.

Slide 19: Is A* Optimal?

This slide illustrates a scenario where A fails to find the optimal solution because the heuristic overestimates the true cost. Although the actual path S-A-G1 costs 4, the algorithm selects the suboptimal path S-G2 (cost 5) because its estimated total cost (5) is lower than the estimated cost of the better path through A (7). The example demonstrates that if the heuristic is not admissible, A may expand and terminate with a non-optimal solution.

Slide 20: Admissible Heuristics

An admissible heuristic never overestimates the true cheapest remaining cost to a goal. This lower-bound property is the key condition behind A* optimality in tree search.

Slide 21: Admissible Heuristics

An admissible heuristic $h(n)$ must satisfy $0 \le h(n) \le h^(n)$, where $h^(n)$ is the true cost to the nearest goal, ensuring the estimate is always optimistic or exact. This property prevents overestimating distances, which is critical for guaranteeing optimal solutions in search algorithms. For instance, Manhattan distance serves as an admissible heuristic by relaxing the problem constraints, such as ignoring walls, thereby calculating a path that is either equal to or shorter than the actual cost required when obstacles exist.

Slide 22: Optimality of A* Tree Search

A* tree search is optimal when step costs are nonnegative and the heuristic is admissible. Tree search may revisit a state along different paths, so its guarantee differs from graph search with a closed set.

Slide 23: Optimality of A* Tree Search (no loop br.)

This section introduces a proof demonstrating that A* tree search without loop checking guarantees an optimal solution when the heuristic h is admissible. Given an optimal goal node A and a suboptimal goal node B, the objective is to prove that A will be selected for expansion before B. The professor outlines that this proof proceeds by analyzing specific cases to show that the algorithm always prioritizes the optimal goal node over any suboptimal alternative.

Slide 24: Optimality of A* Tree Search: Blocking

To prove A tree search optimality, we show that an ancestor node n of the optimal goal A is expanded before any suboptimal goal B. Since the heuristic h is admissible ($h(n) \le h(n)$), the cost function satisfies $f(n) = g(n) + h(n) \le g(n) + h(n)$. Because g(A) equals f(A) at a goal, this inequality establishes that $f(n) \le f(A)$, ensuring n is selected for expansion prior to B*.

Slide 25: Optimality of A* Tree Search: Blocking

To prove A* tree search optimality, consider the critical case where a suboptimal goal node B is on the frontier while the optimal goal A is not. The proof relies on an ancestor n of A also being on the frontier, which must be expanded before B because $f(n) \le f(A)$ and $f(A) < f(B)$. This ensures the algorithm selects the path toward the optimal solution rather than prematurely expanding the suboptimal node B.

Slide 26: Optimality of A* Tree Search: Blocking

To prove A* tree search optimality, assume a suboptimal goal B and an optimal goal A are on the frontier. Since some ancestor n of A must also be on the frontier, and $f(n) \le f(A) < f(B)$, the algorithm expands n before B. This ensures all ancestors of A and A itself are expanded before any suboptimal node B, guaranteeing that the optimal solution is found first.

Slide 27: UCS vs A* Contours

Uniform Cost Search (UCS) expands nodes equally in all directions from the start. In contrast, A primarily expands nodes toward the goal while hedging its bets to guarantee an optimal solution. This directional focus distinguishes A's search contour from UCS's uniform expansion pattern.

Slide 28: Comparison

Slide 28 compares three search strategies based on their evaluation functions: Greedy Best-First Search uses the heuristic estimate h, Uniform Cost Search relies solely on the path cost g, and A* combines both using the sum $g+h$. This distinction highlights how each algorithm prioritizes nodes during the search process.

Slide 29: A* Applications

The A* algorithm is applied across diverse domains including video games, pathing and routing problems, resource planning, and robot motion planning. It also supports language analysis tasks such as machine translation and speech recognition, alongside scientific applications like protein design and chemical synthesis.

Slide 30: Creating Heuristics

A useful heuristic can come from a relaxed version of the problem that removes constraints, making the relaxed solution cost a lower bound on the real solution. For route finding, straight-line distance is a familiar lower bound on road distance. A more informed admissible heuristic is closer to the true remaining cost while never exceeding it.

Slide 31: Creating Admissible Heuristics

Admissible heuristics are often derived from solutions to relaxed problems where additional actions are available. A problem P2 is a relaxed version of P1 if its action set A2(s) includes all actions in A1(s) for every state s. Consequently, the optimal cost h2(s) in the relaxed problem is less than or equal to the optimal cost h1(s) in the original problem, ensuring h2*(s) serves as an admissible heuristic for P1.

Slide 32: Example: 8 Puzzle

In the 8 Puzzle example, a state is defined as the specific location of every square on the grid, such as the sequence "7, 2, 4, 5, empty, 6, 8, 3, 1." The total number of states encompasses all possible combinations of these numbers across the grid positions. This slide introduces the start and goal configurations while posing key questions regarding the definition of states, their quantity, available actions, step costs, and potential heuristics.

Slide 33: 8 Puzzle I

Slide 33 introduces the "Number of tiles misplaced" heuristic for the 8 Puzzle, noting its admissibility. Data from Andrew Moore demonstrates that using this heuristic (ATILES) drastically reduces average nodes expanded compared to Uniform Cost Search (UCS) as optimal path length increases from 4 to 12 steps. For instance, at 12 steps, ATILES expands only 227 nodes versus 3.6 million for UCS.

Slide 34: 8 Puzzle II

This slide introduces a relaxed version of the 8-puzzle where tiles can move freely in any direction, ignoring obstacles. The total Manhattan distance serves as an admissible heuristic for this easier problem because it represents the minimum moves required under these relaxed constraints. Empirical data shows that using the Manhattan distance heuristic (AMANHATTAN) expands significantly fewer nodes than the tile-based heuristic (ATILES) across optimal path lengths of 4, 8, and 12 steps.

Slide 35: 8 Puzzle III

Using the actual cost as a heuristic in A* creates a trade-off between estimate quality and computational work per node. While heuristics closer to the true cost reduce the number of expanded nodes, they typically require more effort to compute. Additionally, combining multiple admissible heuristics by taking their maximum yields a better, more accurate optimistic estimate for each node.

Slide 36: Trivial Heuristics, Dominance

Admissible heuristics form a semi-lattice based on dominance, where heuristic h_a dominates h_c if $h_a(n) \ge h_c(n)$ for all nodes, providing more accurate estimates that prioritize promising candidates. The maximum of admissible heuristics remains admissible and improves estimation accuracy. This lattice ranges from the zero heuristic at the bottom, which reduces search to Uniform Cost Search, to the exact heuristic at the top, though the latter may fail optimality in graphs containing loops.

Slide 37: Optimality of A* Graph Search (w. closed set loop breaking)

With a closed set, A* graph search needs a consistent heuristic, or a way to reopen states when a cheaper path is found. Consistency requires h(n) ≤ c(n,n′) + h(n′), making f(n)=g(n)+h(n) nondecreasing along a path.

Slide 38: A* Graph Search Gone Wrong?

Slide 38 illustrates a scenario where A* Graph Search fails to find the optimal path because node C is prematurely added to the 'closed' set. Although a shorter path to C exists via node B, the algorithm rejects it since C was already visited through a longer route from A. Consequently, the search tree expands suboptimal paths, such as reaching G with a cost of 6 instead of the optimal cost of 5, demonstrating how loop-breaking mechanisms can prevent re-evaluating better routes in graph search.

Slide 39: Alternative: Consistency of Heuristics

Consistency requires that the heuristic "arc" cost between nodes does not exceed the actual arc cost, satisfying the triangle inequality $h(A) \le c(A,C) + h(C)$. Unlike admissibility, which bounds the total cost to the goal, consistency ensures this condition holds for every individual step. A key consequence is that the evaluation function $f(n) = g(n) + h(n)$ never decreases along any path, guaranteeing $f(A) \le f(AC)$ for successor nodes.

Slide 40: A* Graph Search Gone Wrong?

Slide 40 contrasts two state space graphs for A graph search to illustrate the impact of heuristic consistency. The first graph uses an inconsistent heuristic (h=4 at node C), while the second modifies this value to h=2, rendering the heuristic consistent. The slide concludes by assigning an exercise to write out the resulting A search tree based on these configurations.

Slide 41: Optimality of A* Graph Search

A graph search is optimal when using a consistent heuristic, as demonstrated by two key facts: first, A expands nodes in increasing order of total f-value (f-contours); second, for any state, nodes reaching it optimally are expanded before suboptimal paths because the f-value along a path never decreases. This ensures that the algorithm finds the optimal solution without revisiting states with higher costs.

Slide 42: Optimality

For tree search, A is optimal if the heuristic is admissible. In graph search using a closed set, A requires a consistent heuristic to guarantee optimality. Since consistency implies admissibility and most natural heuristics derived from relaxed problems are consistent, this condition is frequently satisfied in practice.

Slide 43: Upgrade: optimal A* with admissible heuristic

This slide upgrades A* to ensure optimality with admissible heuristics by modifying the closed set to store both expanded states and their costs. The algorithm now expands a state if it is not in the closed dictionary or if the current path cost $g(n)$ is lower than the previously recorded cost, updating the dictionary accordingly. While this approach relaxes heuristic design constraints, it risks expanding the same state multiple times if the heuristic is poor.

Slide 44: But…

A is memory-intensive because it stores the entire explored region, often exhausting space before finding a solution. To address this, IDA uses an f-limit instead of depth, RBFS employs recursive depth-first search with ancestor-based limits, and SMA* utilizes all available memory by discarding the worst nodes while retaining their values in parent nodes.

Slide 45: A*: Summary

A* orders nodes in the search queue using the evaluation function f(n) = g(n) + h(n). The algorithm guarantees optimality for both trees and graphs when employing admissible or consistent heuristics. Effective heuristic design is critical, often achieved by solving relaxed versions of the original problem.

Key points to remember

The lecture culminates in the introduction of A search, which combines UCS and GBFS by evaluating nodes using $f(n) = g(n) + h(n)$. A guarantees an optimal solution in tree search if the heuristic is admissible (never overestimating the true cost), a property often achieved by solving relaxed versions of the problem, such as ignoring obstacles in the 8-puzzle. For graph search involving loop-breaking via a closed set, optimality requires the stronger condition of consistency, where the heuristic satisfies the triangle inequality to ensure $f(n)$ values never decrease along a path. While A is powerful and widely applied in domains ranging from robotics to language analysis, it is memory-intensive, prompting the discussion of space-efficient variants like IDA, RBFS, and SMA*.